

Filières d'ingénieurs

Module : Développement Web avec Django

Projet de Fin de Module

Plateforme de Gestion de Rendez-vous Médicaux

MediBook Django : développement, sécurisation et déploiement d'une plateforme intelligente de gestion de rendez-vous médicaux

Niveau : Filières d'ingénieurs

Type : Projet de fin de module

Technologie principale : Django

Travail : Individuel ou en binôme

Livrables : Code source, rapport, démonstration, déploiement

Année universitaire : 2025 – 2026

1. Intitulé du projet

MediBook Django : développement, sécurisation et déploiement d'une plateforme intelligente de gestion de rendez-vous médicaux

Le projet consiste à concevoir et développer une application web de gestion de rendez-vous médicaux avec le framework Django.

La plateforme doit permettre aux patients de rechercher des médecins, consulter leurs disponibilités, réserver des rendez-vous et suivre l'historique de leurs consultations. Elle doit également permettre aux médecins ou aux responsables médicaux de gérer leurs créneaux, leurs rendez-vous et les informations liées aux consultations.

L'objectif du projet n'est pas de remplacer une consultation médicale réelle, mais de développer une application web professionnelle permettant d'améliorer l'organisation des rendez-vous et la gestion administrative dans un contexte médical.

2. Contexte général

La prise de rendez-vous médicaux est un processus important dans les cliniques, cabinets médicaux, centres de santé et établissements hospitaliers. Dans plusieurs situations, la gestion manuelle des rendez-vous peut provoquer des problèmes tels que :

- surcharge des appels téléphoniques ;
- difficulté à connaître les créneaux disponibles ;
- erreurs dans la planification ;
- annulations non suivies ;
- temps d'attente important ;
- absence d'un historique clair des rendez-vous ;
- difficulté à organiser les agendas des médecins.

Dans ce contexte, une plateforme web de gestion de rendez-vous médicaux peut faciliter la relation entre les patients, les médecins et l'administration.

Ce projet permet aux étudiants de mettre en pratique plusieurs notions importantes du développement web :

- conception d'une application Django complète ;
- modélisation d'une base de données relationnelle ;
- gestion des utilisateurs et des rôles ;
- gestion des disponibilités et des créneaux horaires ;
- réservation, annulation et suivi des rendez-vous ;

- tableaux de bord ;
- sécurité et confidentialité des données ;
- intégration d'une fonctionnalité basée sur l'intelligence artificielle ;
- conteneurisation avec Docker ;
- déploiement et CI/CD.

3. Problématique

Problématique du projet

Comment concevoir, développer, sécuriser et déployer une plateforme web de gestion de rendez-vous médicaux avec Django, tout en intégrant une fonctionnalité intelligente permettant d'orienter les patients et d'améliorer l'organisation des rendez-vous ?

Le projet doit répondre aux questions suivantes :

- comment gérer différents types d'utilisateurs : patient, médecin, administrateur ?
- comment organiser les disponibilités des médecins ?
- comment éviter les conflits de réservation ?
- comment permettre au patient de réserver, modifier ou annuler un rendez-vous ?
- comment assurer la confidentialité des informations médicales ?
- comment intégrer une fonctionnalité IA utile sans faire de diagnostic médical automatique ?
- comment déployer l'application dans un environnement proche de la production ?
- comment automatiser les tests et la construction du projet avec une pipeline CI/CD ?

4. Objectifs du projet

4.1. Objectif général

L'objectif général de ce projet est de développer une plateforme web avec Django permettant la gestion des rendez-vous médicaux entre patients et médecins, avec une fonctionnalité intelligente d'aide à l'orientation ou à la recommandation de spécialité.

4.2. Objectifs spécifiques

À la fin du projet, l'étudiant doit être capable de :

- concevoir une architecture Django propre et modulaire ;

- développer une application web avec plusieurs rôles utilisateurs ;
- gérer l'inscription et l'authentification des patients ;
- gérer les profils des médecins ;
- gérer les spécialités médicales ;
- gérer les disponibilités et les créneaux horaires ;
- permettre la réservation d'un rendez-vous ;
- éviter les conflits de réservation ;
- permettre l'annulation ou la modification d'un rendez-vous ;
- créer un tableau de bord pour les médecins et l'administration ;
- intégrer une fonctionnalité d'intelligence artificielle ;
- appliquer les bonnes pratiques de sécurité web ;
- conteneuriser l'application avec Docker ;
- configurer Docker Compose ;
- mettre en place une pipeline CI/CD avec GitHub Actions ;
- documenter et présenter le projet.

5. Acteurs de l'application

L'application doit gérer au minimum quatre types d'utilisateurs.

5.1. Visiteur

Le visiteur est un utilisateur non authentifié. Il peut :

- consulter la page d'accueil ;
- consulter la liste des médecins ;
- rechercher un médecin par nom ou spécialité ;
- consulter les spécialités disponibles ;
- créer un compte patient.

5.2. Patient

Le patient est un utilisateur authentifié. Il peut :

- se connecter à son compte ;
- modifier son profil ;
- rechercher un médecin ;
- filtrer les médecins par spécialité ;
- consulter les disponibilités d'un médecin ;

- réserver un rendez-vous ;
- modifier ou annuler un rendez-vous selon les règles définies ;
- consulter l'historique de ses rendez-vous ;
- recevoir des notifications ou rappels ;
- donner éventuellement un avis sur la qualité du service.

5.3. Médecin

Le médecin est un utilisateur authentifié avec un rôle spécifique. Il peut :

- gérer son profil professionnel ;
- définir ses spécialités ;
- gérer ses disponibilités ;
- consulter la liste de ses rendez-vous ;
- confirmer ou annuler un rendez-vous ;
- ajouter un résumé administratif de consultation ;
- consulter son planning journalier ou hebdomadaire ;
- accéder à des statistiques simples liées à ses rendez-vous.

5.4. Administrateur

L'administrateur est responsable de la gestion globale de la plateforme. Il peut :

- gérer les comptes utilisateurs ;
- gérer les médecins ;
- gérer les spécialités ;
- consulter l'ensemble des rendez-vous ;
- gérer les paramètres de la plateforme ;
- consulter les statistiques générales ;
- contrôler les données et la sécurité de l'application.

6. Fonctionnalités attendues

6.1. Gestion des comptes utilisateurs

L'application doit permettre :

- l'inscription des patients ;
- la connexion et la déconnexion ;

- la modification du profil utilisateur ;
- la gestion des rôles ;
- la protection des pages selon le rôle ;
- la réinitialisation du mot de passe, si possible.

6.2. Gestion des médecins

Chaque médecin doit avoir un profil contenant au minimum :

- nom et prénom ;
- email ;
- téléphone professionnel ;
- spécialité ;
- adresse du cabinet ou centre médical ;
- description courte ;
- années d'expérience, si nécessaire ;
- photo de profil, si possible ;
- statut actif ou inactif.

6.3. Gestion des spécialités médicales

L'application doit permettre la gestion des spécialités médicales.

Exemples de spécialités :

- médecine générale ;
- cardiologie ;
- dermatologie ;
- pédiatrie ;
- gynécologie ;
- ophtalmologie ;
- dentisterie ;
- ORL ;
- neurologie ;
- radiologie.

6.4. Gestion des disponibilités

Le médecin doit pouvoir définir ses disponibilités :

- jours de travail ;

- heure de début ;
- heure de fin ;
- durée moyenne d'un rendez-vous ;
- périodes d'indisponibilité ;
- jours de congé.

Le système doit empêcher la réservation de deux rendez-vous sur le même créneau pour le même médecin.

6.5. Réservation des rendez-vous

Le patient doit pouvoir :

- sélectionner une spécialité ;
- choisir un médecin ;
- consulter les créneaux disponibles ;
- choisir une date et une heure ;
- préciser le motif du rendez-vous ;
- confirmer la réservation ;
- recevoir une confirmation.

6.6. Gestion des rendez-vous

Un rendez-vous doit contenir au minimum :

- le patient concerné ;
- le médecin choisi ;
- la spécialité ;
- la date ;
- l'heure ;
- le motif ;
- le statut ;
- la date de création.

Les statuts possibles sont :

- en attente ;
- confirmé ;
- annulé ;
- terminé ;
- absent.

6.7. Tableau de bord patient

Le tableau de bord patient doit afficher :

- les prochains rendez-vous ;
- les rendez-vous passés ;
- les rendez-vous annulés ;
- les informations du profil ;
- les notifications importantes.

6.8. Tableau de bord médecin

Le tableau de bord médecin doit afficher :

- les rendez-vous du jour ;
- les rendez-vous de la semaine ;
- le nombre total de rendez-vous ;
- les rendez-vous confirmés ;
- les rendez-vous annulés ;
- les patients à recevoir ;
- le planning du médecin.

6.9. Tableau de bord administrateur

Le tableau de bord administrateur doit afficher :

- nombre total de patients ;
- nombre total de médecins ;
- nombre total de rendez-vous ;
- nombre de rendez-vous par spécialité ;
- nombre de rendez-vous par statut ;
- spécialités les plus demandées ;
- médecins les plus sollicités.

7. Fonctionnalité d'intelligence artificielle

Fonctionnalité IA obligatoire

Chaque projet doit intégrer au minimum une fonctionnalité basée sur l'intelligence artificielle. Pour ce sujet, la fonctionnalité recommandée est une aide intelligente à l'orientation vers une spécialité médicale.

7.1. Aide intelligente à l'orientation vers une spécialité

La plateforme peut intégrer un module intelligent qui aide le patient à choisir une spécialité médicale en fonction du motif de consultation saisi.

Exemple :

- si le patient saisit : *douleur thoracique, essoufflement, palpitations*, le système peut suggérer la cardiologie ;
- si le patient saisit : *problème de peau, boutons, rougeurs*, le système peut suggérer la dermatologie ;
- si le patient saisit : *douleur dentaire*, le système peut suggérer la dentisterie ;
- si le patient saisit : *fièvre chez un enfant*, le système peut suggérer la pédiatrie.

Limite importante

La fonctionnalité IA ne doit pas faire un diagnostic médical. Elle doit uniquement proposer une orientation indicative vers une spécialité. La décision finale appartient toujours au patient, au médecin ou au personnel médical.

7.2. Approche technique proposée

Pour rester réaliste dans un projet Django, les étudiants peuvent utiliser une approche simple basée sur :

- une liste de mots-clés associés aux spécialités ;
- une représentation textuelle du motif de consultation ;
- TF-IDF pour représenter les textes ;
- similarité cosinus pour comparer le motif du patient avec les descriptions des spécialités ;
- scikit-learn pour l'implémentation.

7.3. Exemple de fonctionnement

Le patient saisit le motif suivant :

J'ai des douleurs au niveau de la poitrine et je ressens parfois des palpitations.

Le système analyse le texte et propose :

- spécialité recommandée : cardiologie ;
- autres suggestions possibles : médecine générale.

7.4. Variantes avancées

Les groupes avancés peuvent proposer d'autres fonctionnalités IA :

- chatbot simple d'orientation administrative ;
- prédiction des créneaux les plus demandés ;
- recommandation du médecin selon la spécialité et la disponibilité ;
- résumé automatique du motif de consultation ;
- classification automatique des rendez-vous selon l'urgence administrative ;
- analyse des annulations et absences.

8. Technologies recommandées

Partie	Technologies recommandées
Backend	Django
API REST optionnelle	Django REST Framework
Frontend	Django Templates avec Bootstrap ou Tailwind CSS
Base de données	PostgreSQL ou MySQL
Authentification	Django Authentication System
Gestion des rôles	Groups et Permissions de Django
Intelligence artificielle	Python, scikit-learn, TF-IDF, similarité cosinus
Gestion des dates	DateTimeField, TimeField, logique de créneaux
Tests	Django TestCase ou Pytest
Conteneurisation	Docker
Orchestration locale	Docker Compose
Serveur de production	Gunicorn
Reverse proxy	Nginx
CI/CD	GitHub Actions
Dépôt distant	GitHub
Registre d'images	Docker Hub ou GitHub Container Registry

9. Architecture proposée

L'application peut être organisée en plusieurs applications Django.

Architecture indicative

L'organisation suivante est proposée à titre indicatif. Chaque groupe peut l'adapter selon ses besoins.

```
medibook_project/
|
|-- accounts/
|   |-- gestion des utilisateurs, rôles, inscription, connexion
|
|-- patients/
|   |-- gestion des profils patients
|
|-- doctors/
|   |-- gestion des médecins, spécialités et profils médicaux
|
|-- appointments/
|   |-- gestion des rendez-vous et des créneaux
|
|-- schedules/
|   |-- gestion des disponibilités des médecins
|
|-- dashboard/
|   |-- statistiques et tableaux de bord
|
|-- ai_orientation/
|   |-- fonctionnalité IA d'orientation vers une spécialité
|
|-- notifications/
|   |-- rappels et notifications
|
|-- templates/
|
|-- static/
|
|-- media/
|
|-- Dockerfile
|
```

```
|-- docker-compose.yml
|
|-- requirements.txt
|
|-- .env
|
|-- .github/
    |-- workflows/
        |-- ci-cd.yml
```

10. Déploiement du projet

10.1. Déploiement local avec Docker Compose

Le projet doit être exécutable localement avec Docker Compose.

L'environnement Docker doit contenir au minimum :

- un service pour l'application Django ;
- un service pour la base de données PostgreSQL ou MySQL ;
- éventuellement un service Nginx ;
- un fichier `.env` pour les variables d'environnement.

10.2. Variables d'environnement

Les informations sensibles ne doivent pas être écrites directement dans le code source.

Exemples de variables à placer dans un fichier `.env` :

```
DEBUG=False
SECRET_KEY=your_secret_key
DB_NAME=medibook_db
DB_USER=medibook_user
DB_PASSWORD=your_password
DB_HOST=db
DB_PORT=5432
ALLOWED_HOSTS=localhost,127.0.0.1
```

10.3. Déploiement en production

Pour la version finale, le projet peut être déployé sur :

- une machine virtuelle Ubuntu ;
- Oracle Cloud ;
- Render ;
- Railway ;
- un VPS ;
- un serveur local de test.

La version de production doit respecter les points suivants :

- désactiver le mode debug avec `DEBUG=False` ;
- utiliser une clé secrète stockée dans `.env` ;
- utiliser Gunicorn comme serveur WSGI ;
- utiliser Nginx comme reverse proxy, si possible ;
- utiliser PostgreSQL ou MySQL comme base de données ;
- gérer correctement les fichiers statiques ;
- protéger les variables sensibles ;
- configurer correctement `ALLOWED_HOSTS` ;
- éviter l'exposition des informations sensibles.

11. CI/CD avec GitHub Actions

Objectif de la CI/CD

La pipeline CI/CD doit automatiser les tests, la vérification du projet, la construction de l'image Docker et éventuellement le déploiement.

11.1. Pipeline minimale attendue

À chaque `push` sur la branche principale, la pipeline doit :

1. récupérer le code source depuis GitHub ;
2. installer Python ;
3. installer les dépendances du projet ;
4. lancer les migrations en environnement de test ;
5. exécuter les tests ;
6. vérifier la qualité du code ;
7. construire l'image Docker ;
8. publier l'image Docker sur Docker Hub ou GitHub Container Registry.

11.2. Pipeline avancée

Les groupes avancés peuvent ajouter :

- un scan de sécurité des dépendances ;
- un scan de sécurité de l'image Docker ;
- une séparation entre environnement de développement, test et production ;
- un déploiement automatique vers une machine virtuelle ;
- une notification en cas d'échec de la pipeline ;
- l'exécution automatique des tests de sécurité.

12. Exigences de sécurité et confidentialité

Importance de la confidentialité

Le projet traite des données liées à des patients et à des rendez-vous médicaux. Les étudiants doivent donc appliquer des règles strictes de sécurité et de confidentialité.

Le projet doit respecter les bonnes pratiques suivantes :

- l'authentification est obligatoire pour réserver un rendez-vous ;
- chaque utilisateur doit accéder uniquement à ses propres données ;
- un patient ne doit pas consulter les rendez-vous d'un autre patient ;
- un médecin doit consulter uniquement ses propres rendez-vous ;
- les mots de passe doivent être gérés par le système d'authentification de Django ;
- la protection CSRF doit rester activée ;
- les formulaires doivent être validés côté serveur ;
- les pages d'administration doivent être protégées ;
- les variables sensibles doivent être placées dans un fichier `.env` ;
- le mode debug doit être désactivé en production ;
- les erreurs doivent être gérées correctement ;
- les données personnelles doivent être minimisées ;
- les informations médicales sensibles doivent être évitées ou limitées dans le cadre du projet pédagogique.

Attention

Il est interdit de stocker les mots de passe en clair dans la base de données. Il est également interdit de publier le fichier `.env` sur GitHub.

13. Modèle de données minimal

Le modèle de données peut contenir au minimum les entités suivantes :

Entité	Description
Utilisateur	Représente les utilisateurs de la plateforme : patient, médecin ou administrateur.
ProfilPatient	Contient les informations complémentaires du patient.
Médecin	Contient les informations professionnelles du médecin.
Spécialité	Représente une spécialité médicale.
Disponibilité	Représente les jours et heures disponibles d'un médecin.
Créneau	Représente un créneau horaire reservable.
RendezVous	Représente un rendez-vous entre un patient et un médecin.
Consultation	Représente un résumé administratif après le rendez-vous.
Notification	Représente un rappel ou une information envoyée à l'utilisateur.
Avis	Représente un avis optionnel donné par un patient sur le service.

14. Livrables attendus

Chaque groupe doit fournir les livrables suivants :

1. code source complet sur GitHub ;
2. rapport détaillé du projet ;
3. diagramme de cas d'utilisation ;
4. diagramme de classes ;
5. modèle de données ;
6. captures d'écran de l'application ;
7. description de la fonctionnalité IA ;
8. fichier `Dockerfile` ;
9. fichier `docker-compose.yml` ;
10. fichier de pipeline GitHub Actions ;
11. démonstration fonctionnelle de l'application ;
12. courte vidéo de démonstration ;
13. présentation finale.

15. Contenu du rapport

Le rapport doit contenir les parties suivantes :

- introduction générale ;
- contexte du projet ;
- problématique ;
- objectifs ;
- analyse des besoins ;
- identification des acteurs ;
- diagramme de cas d'utilisation ;
- diagramme de classes ;
- architecture technique ;
- choix technologiques ;
- description des fonctionnalités ;
- description de la fonctionnalité IA ;
- mesures de sécurité et confidentialité ;
- description du déploiement ;
- description de la pipeline CI/CD ;
- captures d'écran ;
- difficultés rencontrées ;
- conclusion et perspectives.

16. Travail attendu

L'étudiant ou le groupe doit réaliser une application complète et fonctionnelle. Le projet doit permettre de gérer le parcours complet d'un rendez-vous médical, depuis la recherche d'un médecin jusqu'à la confirmation du rendez-vous.

Le projet doit être suffisamment professionnel pour montrer la maîtrise :

- de Django ;
- de la conception d'applications web ;
- de la base de données ;
- de l'authentification ;
- de la gestion des rôles ;
- de la sécurité ;
- de la confidentialité ;
- du déploiement ;

- de la CI/CD ;
- et de l'intégration d'une fonctionnalité intelligente.